

Tenbin Audit Scope 8/03/26

Commit Hashes

There are 2 commit hashes in 2 different repos that make up this scope. The cross chain cooldown feature can be reviewed as a diff versus the original code - only code that has been modified needs to be in scope. Total lines of code: ~ 569.

- Tenbin contracts: [633540ee51d7a069f4a5a53feccd8e3498d1ff14](#)
- Cross chain cooldown: [cde2faffec4470c9a3f33103efd6a52d027cd696](#)

Code In Scope

Core Contracts

- [633540ee51d7a069f4a5a53feccd8e3498d1ff14](#)
- ~420 lines added
- src/external/morpho/GenericMorphoAdapter.sol
- src/external/morpho/GenericMorphoAdapterFactory.sol
- src/external/bebop/BebopHook.sol
- src/external/bebop/IBebopHook.sol

Cross chain cooldown

- [cde2faffec4470c9a3f33103efd6a52d027cd696](#)
- ~149 lines added, ~233 lines removed
- src/CrossChainERC4626.sol
- Note: this is a fork of a contract already audited by chainlink.

Contact

Tenbin Labs CEO

yuki@tenbinlabs.xyz

Smart Contract Engineers

hal@tenbinlabs.xyz

julissa@tenbinlabs.xyz

Audit Objectives

Goals

- Review the cross chain cooldown commit versus the original code
- Validate cross chain cooldown contract is secure in the context of Chainlink CCIP
- Ensure cross chain operations do not result in stuck funds
- Validate Bebop hook contract is secure in the context of the Bebop ecosystem
- Review GenericMorphoAdapter and ensure implementation is compatible with Morpho V2 vaults and introduces minimal risk
- Ensure GenericMorphoAdapter can act as a Vault V2 adapter which funds any standard, non-synchronous ERC4626 vault

Assumptions

- Existing CrossChainERC4626 pre-fork is secure according to Chainlink's standards. Audit was performed by
- GenericMorphoAdapter will only be used to fund standard, non synchronous ERC4626 vaults
- GenericMorphoAdapter will not be used fee-on-transfer ERC20 tokens

External Dependencies

- Bebop Hook :
<https://github.com/bebop-dex/bebop-rfqa/blob/master/contracts/interfaces/IBebopHook.sol>
- Bebop Router :
<https://github.com/bebop-dex/bebop-rfqa/blob/master/contracts/BebopRouter.sol>
- Chainlink CCIP :
<https://github.com/smartcontractkit/chainlink-ccip/blob/main/chains/evm/contracts>
- Morpho V2 Vault : <https://github.com/morpho-org/vault-v2/>

Audit Overview

Tenbin is an asset token issuance platform with the goal of creating liquid, composable financial assets. Assets in the Tenbin protocol are backed by two positions: off-chain futures contracts and on-chain collateral. The off-chain hedging system maintains a delta one exposure of an underlying asset. The on-chain collateral is used to earn low-risk yield. So long as the on-chain yield equals or exceeds the off-chain funding costs, the protocol is able to peg Tenbin assets to the spot price of the real asset.

CrossChainERC4626: 149 lines added, 233 lines removed

BebopHook: 241 lines

IBebopHook: 20 lines

GenericMorphoAdapter: 129 lines

GenericMorphoAdapterFactory: 30 lines

Total: ~569 lines added

GenericMorphoAdapter

GenericMorphoAdapter is an Adapter that connects a Vault V2 parent vault to a single ERC4626 vault. The parent vault allocates the underlying asset to this adapter, and the adapter deposits those assets into the configured ERC4626 vault. Accounting is reported back to the parent vault through a single adapter-specific allocation ID.

This adapter was developed because Morpho does not have an audited generic ERC4626 adapter available. This adapter is a modification of the Morpho adapter here:

<https://github.com/morpho-org/vault-v2/blob/main/src/adapters/MorphoVaultV1Adapter.sol>

BebopHook

BebopHook is an adapter between Bebop's RFQ router and a Tenbin Controller. It makes a Tenbin mint or redemption look like a Bebop

swap while sourcing the output through Tenbin's primary market instead of the market maker's inventory.

The hook holds no order state: each order either completes atomically or reverts. Replay protection, expiry, signer authorization, and order limits remain in the Bebop router and Tenbin controller.

Participants

- User/taker: supplies the input token and calls the Bebop router.
- Bebop router: transfers the user's input, grants the hook an exact per-execution allowance, and invokes the hook.
- Market maker: signs the Bebop hook/PMM data and the Tenbin controller order.
- BebopHook: validates that both order systems describe the same conversion, then calls the controller.
- Controller: verifies the order and minter approval, consumes its nonce, and performs the actual mint or redemption.

Each deployed hook is permanently bound to one router and one controller. Its owner can rotate the market maker, manage controller delegates, and rescue residual tokens.

Execution

The router calls `bebopHook(makerAddress, data, swaps)`. The hook then:

1. Requires the configured router, configured market maker, and exactly one swap leg.
2. Decodes:
 - HookData: action, token pair, signed quote amounts, and controller data.
 - IssuerData: controller order, order signature, context, and minter approval.
3. Reads the actual filled input from the router's allowance to the hook. Bebop's `needsApproval` behavior sets this allowance to the exact fill amount.
4. Requires that amount to equal the fixed-size controller order:
 - Mint: `collateral_amount`

- Redeem: asset_amount
- 5. Cross-checks the action, tokens, amounts, payer, recipient, and signed rate.
- 6. Pulls the input from the router and gives the controller an exact allowance.
- 7. Call controller.mint() or controller.redeem().
- 8. Measures how much output was actually produced, requires at least the promised amount, and transfers that amount back to the router.

Cross Chain Cooldown

CrossChainERC4626Adapter is a fork of Chainlink's cross chain ERC4626 adapter. The original adapter is designed to allow deposit and redemption of ERC4626 tokens across different chains.

This contract can be analyzed as a diff versus the original code base. The original version pre-fork has already been audited.

High Level Changes

- The original adapter was copied to src/CrossChainERC4626AdapterLegacy.sol.
- src/CrossChainERC4626Adapter.sol became a single-vault adapter supporting Tenbin cooldowns and unstaking.
- Local interface files were added to support tenbin interactions
- New tests were added to test custom logic
- Existing scripts, factory tests, and README references were redirected to the legacy adapter.
- The Tenbin-specific adapter removes target allowlisting and flat adapter-fee accounting in favor of an immutable vault and asset.
- At this commit, the payload remains 128 bytes, but its first word changes from a vault target address to a packed action/cooldown ID. The two formats are therefore semantically incompatible despite having the same length.

Intended Tenbin Interactions

The supplied vault must implement both ERC-4626 and Tenbin's `IStakedAsset`. The adapter stores that vault and its underlying asset as immutables. Tenbin sees the adapter as the owner of all vault positions and cooldowns; the adapter separately tracks which remote account controls each cooldown.

The payload packs an action into bits 128–135 and a uint128 cooldown ID into the lower 128 bits:

- 0 - DEPOSIT: CCIP delivers the underlying asset. The adapter calls `vault.deposit()`, then delivers or bridges the resulting staking shares to the beneficiary.
- 1 - COOLDOWN: CCIP delivers staking shares. The adapter calls `cooldownShares()`. Tenbin burns the shares, moves the corresponding underlying into its silo, and creates a cooldown owned by the adapter. The adapter records (source chain, cooldown ID) → original sender and emits `CrossChainCooldownCreated`. No token is returned during this step; callers learn the ID from the event.
- 2 - UNSTAKE: After the cooldown expires, the same sender on the same source chain sends a tokenless message containing the cooldown ID. The adapter reads the cooldown amount, calls `unstake(address(this), id)`, and then locally delivers or bridges the released underlying asset.
- 3 - CANCEL_COOLDOWN: The same authorization check applies. Tenbin moves the underlying out of the silo and mints replacement staking shares to the adapter, which then delivers or bridges those shares.

Deposit and cooldown-start failures retain the existing failed-message recovery mechanism because those messages carry tokens. Unstake and cancellation are tokenless control messages; if one fails, the user can submit a fresh message after correcting the condition, such as waiting for the cooldown to finish.

END